

of methods contained in source code. In this case, each method can be treated as an independent document and the high level comment associated with that method or function is nothing but a short summary of the method.

Another popular text analysis technique involves the use of language models, which enables predicting what the next word would be in a particular sentence. This technique is typically used in optical character recognition (OCR) generated documents, where due to OCR errors, the next word is not visible or gets lost and hence the tool needs to make a best case estimate of the word that may appear there. A similar need also arises in the case of speech recognition systems. In case of poor speech quality, when a sentence is being transcribed by the speech recognition tool, a particular word may not be clear or could get lost in transmission. In such a case, the tool needs to predict what the missing word is and add it automatically.

Language modelling techniques can also be applied in intelligent development environments (IDE) to provide 'auto-completion' suggestions to the developers. Note that in this case, the source code itself is being treated as text and is analysed.

Classifying a set of documents into specific categories is another well-known text analysis technique. Consider a large number of news articles that need to be categorised based on topics or their genre, such as politics, business, sports, etc. A number of well-known text analysis techniques are available for document classification. Document classification techniques can also be applied to defect reports in SE to classify the category to which the defect belongs. For instance, security related bug reports need to be prioritised. While people currently inspect bug reports, or search for specific key words in a bug category field in Bugzilla reports in order to classify bug reports, more robust and automated techniques are needed to classify defect reports in large scale open source projects. Text analysis techniques for document classification can be employed in such cases.

Another important need in the SE lifecycle is to trace source code to its origin in the requirements document. If a feature 'X' is present in the source code, what is the requirement 'Y' in the requirements document which necessitated the development of this feature? This is known as traceability of source code to requirements. As source code evolves over time, maintaining traceability links automatically through tools is essential to scale out large software projects. Text analysis techniques can be employed to connect a particular requirement from the requirements document to a feature in the source code and hence automatically generate the traceability links.

We have now covered automatic summarisation techniques for generating summaries of bug reports and generating header level comments for methods. Another possible use for such techniques in SE artifacts is to enable the automatic generation of user documentation associated with that software project. A number of text mining techniques have been employed to mine 'stack overflow' mailing lists to generate automatic user documentation or FAQ documents for different software projects.

Regarding the identification of inconsistencies in the requirements document, inconsistency detection techniques can be applied to source code comments also. It is a general expectation that source code comments express the programmer's

intent. Hence, the code written by the developer and the comment associated with that piece of code should be consistent with each other. Consider the simple code sample shown below:

```
/* linux/drivers/scsi/in2000.c: */
/* caller must hold instance lock */
Static int reset_hardware(...)
{
    ...
}

static int in2000_bus_reset(...)
{
    ...
    reset_hardware();
    -
}
```

In the above code snippet, the developer has expressed the intention that 'instance_lock' must be held before the function 'reset_hardware' is called as a code comment. However, in the actual source code, the lock is not acquired before the call to 'reset_hardware' is made. This is a logical inconsistency, which can arise either due to: (a) comments being outdated with respect to the source code; or (b) incorrect code. Hence, flagging such errors is useful to the developer who can fix either the comment or the code, depending on which is incorrect.

My 'must-read book' for this month

This month's book suggestion comes from one of our readers, Sharada, and her recommendation is very appropriate to the current column. She recommends an excellent resource for natural language processing—a book called, 'Speech and Language Processing: An Introduction to Natural Language Processing' by Jurafsky and Martin. The book describes different algorithms for NLP techniques and can be used as an introduction to the subject. Thank you, Sharada, for your valuable recommendation.

If you have a favourite programming book or article that you think is a must-read for every programmer, please do send me a note with the book's name, and a short write-up on why you think it is useful so I can mention it in the column. This would help many readers who want to improve their software skills.

If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming! 

By: Sandya Mannarswamy

The author is an expert in systems software and is currently working with Hewlett Packard India Ltd. Her interests include compilers, multi-core and storage systems. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group 'Computer Science Interview Training India' at <http://www.linkedin.com/groups?home=HYPERLINK&http://www.linkedin.com/groups?home=8&gid=2339182>